ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

Доцент, канд. тех. наук		Степанов П. А.
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №6

Разработка приложения с использованием Hibernate

по курсу: Технология разработки серверных информационных систем

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4434		Энгельгардт К. А.
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1. Текст и вариант задания

Вариант задания: 1 - Книжный магазин либо библиотека.

Задание на лабораторную работу:

- Используйте в качестве основы лабораторную работу номер 4 или 5 (для заочников – 3).
- Добавьте базу данных, модель данных, JPA репозитории и сервисы. Должно быть не менее двух таблиц и одного внешнего ключа.
- Реализуйте операции чтения/записи с помощью JPA (spring-boot-data-jpa, также допустимо spring-boot-data-jdbc если сумеете объяснить разницу).

2. Описание разрабатываемого продукта

В рамках лабораторной работы №6 веб-приложение «Книжный магазин» переведено с временного in-memory хранилища на полнофункциональную реляционную базу данных под управлением объектно-реляционного отображения Hibernate и спецификации Spring Data JPA. При этом полностью сохранены механизмы безопасности, реализованные в лабораторной работе №4: работа по протоколу HTTPS (порт 8443, SSL/TLS сертификат PKCS12), форма аутентификации со стилизацией ГУАП, разделение ролей USER/ADMIN, защита изменяющих эндпоинтов, CSRF-токены и аудит действий.

2.1. Реляционная модель данных и связи сущностей

В соответствии с требованиями задания в базу данных добавлены две взаимосвязанные таблицы, объединенные отношением внешнего ключа (Foreign Key):

- Таблица «publishers» (сущность Publisher) — представляет издательства книг. Содержит поля: id (первичный ключ, автоинкремент), name (наименование издательства, VARCHAR(100)), city (город расположения, VARCHAR(100)), contact_phone (контактный телефон, VARCHAR(50)).
- Таблица «books» (сущность Book) — представляет каталог книг книжного магазина. Содержит поля: id (первичный ключ, автоинкремент), title (название книги, VARCHAR(200)), author (автор, VARCHAR(150)), genre (жанр, VARCHAR(100)), pub_year (год издания, INT), price (цена в рублях, DOUBLE PRECISION), stock (остаток на складе, INT), publisher_id (внешний ключ FK, ссылающийся на id таблицы publishers).

Связь между сущностями организована отношением @ManyToOne со стороны книги (каждая книга выпущена конкретным издательством) и обратным отношением @OneToMany со стороны издательства (одно издательство может выпустить множество книг).

2.2. Применение шаблона проектирования DTO (Data Transfer Object)

Как подчеркивается в методических указаниях к лабораторной работе, прямое использование классов JPA-сущностей во внешнем интерфейсе API является антипаттерном из-за риска передачи на клиент избыточных данных, раскрытия внутренней структуры базы

данных и потенциального падения приложения при циклических связях. Для решения этой проблемы реализован паттерн DTO:

- BookDto — содержит плоскую структуру данных книги, а также идентификатор (publisherId) и наименование издательства (publisherName) без прямого включения вложенных объектов сущностей.
- PublisherDto — объект передачи сведений об издательстве.

Преобразование между сущностями JPA и DTO выполняется в сервисном слое перед отправкой клиенту и при приеме входящих данных.

2.3. Управление транзакциями и репозитории

Доступ к базе данных реализован через Spring Data JPA репозитории BookRepository и PublisherRepository, расширяющие интерфейс JpaRepository. Бизнес-логика вынесена в сервисные классы BookService и PublisherService, методы которых аннотированы @Transactional. Для операций чтения используется режим @Transactional(readOnly = true), оптимизирующий работу сессий Hibernate. При возникновении исключений Spring обеспечивает автоматический откат транзакций. Изменяющие операции сопровождаются логированием в AuditService.

2.4. Стек технологий

- Java 17 (OpenJDK 17 HotSpot)
- Spring Boot 3.2.5 (spring-boot-starter-web, spring-boot-starter-data-jpa)
- Hibernate ORM 6.4.4 (реализация JPA стандарта)
- H2 Database Engine (встроенная реляционная СУБД с web-консолью /h2-console)
- HikariCP (высокопроизводительный пул соединений с базой данных)
- Spring Security (HTTPS на 8443, Form Login, роли USER/ADMIN, аудит)
- Thymeleaf + AngularJS 1.7.5 (клиентское представление с табами каталога и издательств)
- SpringDoc OpenAPI 2.5.0 (документация REST API)

3. Текст основных фрагментов кода

3.1. Конфигурация зависимостей в pom.xml (фрагмент JPA и H2)

```
<!-- Spring Boot Data JPA & Hibernate -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- Встроенная СУБД H2 Database -->
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
```

3.2. Настройки источника данных и Hibernate в application.properties

```
server.port=8443
spring.application.name=bookstore

# SSL / HTTPS (порт 8443)
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=mypassword
server.ssl.keyStoreType=PKCS12
server.ssl.keyAlias=tomcat

# Настройки подключения к СУБД H2 и JPA/Hibernate
spring.datasource.url=jdbc:h2:mem:bookstoredb;DB_CLOSE_DELAY=-
1;DB_CLOSE_ON_EXIT=FALSE
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql=true
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# Аудит и логирование
logging.level.ru.spbguap.bookstore=DEBUG
logging.level.org.springframework.security=INFO
```

3.3. Сущность издательства Publisher.java (первая таблица)

```
@Entity
@Table(name = "publishers")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Publisher {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(length = 100)
    private String city;

    @Column(length = 50)
    private String contactPhone;
```

```
    @OneToMany(mappedBy = "publisher", cascade = CascadeType.ALL, orphanRemoval
= false)
    @Builder.Default
    private List<Book> books = new ArrayList<>();
}
```

3.4. Сущность книги Book.java со связью @ManyToOne (вторая таблица и FK)

```
@Entity
@Table(name = "books")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200)
    private String title;

    @Column(nullable = false, length = 150)
    private String author;

    @Column(length = 100)
    private String genre;

    @Column(name = "pub_year")
    private Integer year;

    @Column(nullable = false)
    private Double price;

    @Column(nullable = false)
    private Integer stock;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "publisher_id")
    private Publisher publisher;
}
```

3.5. Шаблон Data Transfer Object (BookDto.java)

```
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
```

```
public class BookDto {
    private Long id;
    private String title;
    private String author;
    private String genre;
    private Integer year;
    private Double price;
    private Integer stock;
    private Long publisherId;
    private String publisherName;
}
```

3.6. Spring Data JPA репозитории (BookRepository и PublisherRepository)

```
@Repository
public interface BookRepository extends JpaRepository<Book, Long> {
    List<Book> findByAuthorContainingIgnoreCase(String author);
    List<Book> findByGenreIgnoreCase(String genre);
    List<Book> findByPublisherId(Long publisherId);
}

@Repository
public interface PublisherRepository extends JpaRepository<Publisher, Long> {
    Optional<Publisher> findByName(String name);
}
```

3.7. Сервисный слой с поддержкой транзакций (BookService.java)

```
@Service
@RequiredArgsConstructor
@Transactional
public class BookService {

    private final BookRepository bookRepository;
    private final PublisherRepository publisherRepository;
    private final AuditService auditService;

    public BookDto create(BookDto dto) {
        Book book = toEntity(dto);
        Book saved = bookRepository.save(book);
        auditService.logCreate("Book", saved.getId());
        return toDto(saved);
    }

    @Transactional(readOnly = true)
    public List<BookDto> getAll() {
        return
bookRepository.findAll().stream().map(this::toDto).collect(Collectors.toList());
    }

    @Transactional(readOnly = true)
    public Optional<BookDto> getById(Long id) {
```

```
        return bookRepository.findById(id).map(this::toDto);
    }

    public Optional<BookDto> update(Long id, BookDto dto) {
        return bookRepository.findById(id).map(existing -> {
            existing.setTitle(dto.getTitle());
            existing.setAuthor(dto.getAuthor());
            existing.setGenre(dto.getGenre());
            existing.setYear(dto.getYear());
            existing.setPrice(dto.getPrice());
            existing.setStock(dto.getStock());
            if (dto.getPublisherId() != null) {
                Publisher publisher =
publisherRepository.findById(dto.getPublisherId()).orElse(null);
                existing.setPublisher(publisher);
            }
            Book updated = bookRepository.save(existing);
            auditService.logUpdate("Book", updated.getId());
            return toDto(updated);
        });
    }

    public boolean delete(Long id) {
        if (bookRepository.existsById(id)) {
            bookRepository.deleteById(id);
            auditService.logDelete("Book", id, true);
            return true;
        }
        auditService.logDelete("Book", id, false);
        return false;
    }
}
```

3.8. Контроллер REST API для управления книгами (BookRestController.java)

```
@RestController
@RequestMapping("/api/books")
@RequiredArgsConstructor
@Tag(name = "Книги", description = "API для управления каталогом книг (JPA /
Hibernate)")
public class BookRestController {

    private final BookService bookService;

    @PostMapping
    public ResponseEntity<BookDto> createBook(@RequestBody BookDto bookDto) {
        BookDto created = bookService.create(bookDto);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    @GetMapping
    public ResponseEntity<List<BookDto>> getAllBooks() {
        return ResponseEntity.ok(bookService.getAll());
```

```
    }

    @GetMapping("/{id}")
    public ResponseEntity<BookDto> getBookById(@PathVariable Long id) {
        return bookService.getById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    @PutMapping("/{id}")
    public ResponseEntity<BookDto> updateBook(@PathVariable Long id,
@RequestBody BookDto bookDto) {
        return bookService.update(id, bookDto)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteBook(@PathVariable Long id) {
        if (bookService.delete(id)) {
            return ResponseEntity.noContent().build();
        }
        return ResponseEntity.notFound().build();
    }
}
```

3.9. Первичное наполнение данными (DataInitializer.java)

```
@Component
@RequiredArgsConstructor
@Slf4j
public class DataInitializer implements CommandLineRunner {

    private final PublisherRepository publisherRepository;
    private final BookRepository bookRepository;

    @Override
    public void run(String... args) {
        if (publisherRepository.count() == 0) {
            Publisher ast = publisherRepository.save(Publisher.builder()
                    .name("АСТ").city("Москва").contactPhone("+7 (495) 789-45-
00").build());
            Publisher eksmo = publisherRepository.save(Publisher.builder()
                    .name("Эксмо").city("Москва").contactPhone("+7 (495) 411-68-
86").build());
            Publisher azbuka = publisherRepository.save(Publisher.builder()
                    .name("Азбука-Аттикус").city("Санкт-
Петербург").contactPhone("+7 (812) 327-04-55").build());

            bookRepository.save(Book.builder()
                    .title("Война и мир").author("Лев Толстой").genre("Роман")
                    .year(1869).price(590.0).stock(5).publisher(ast).build());
            bookRepository.save(Book.builder()
```

```
                    .title("Преступление и наказание").author("Фёдор
Достоевский").genre("Роман")
                    .year(1866).price(450.0).stock(8).publisher(eksmo).build());
            bookRepository.save(Book.builder()
                    .title("Мастер и Маргарита").author("Михаил
Булгаков").genre("Фантастика")

.year(1967).price(520.0).stock(12).publisher(azbuka).build());
        }
    }
}
```

4. Вывод

В ходе выполнения лабораторной работы №6 веб-сервис «Книжный магазин» был успешно переведён на использование объектно-реляционного отображения Hibernate и технологии Spring Data JPA. Спроектирована и реализована реляционная схема базы данных из двух взаимосвязанных таблиц (publishers и books), объединённых отношением внешнего ключа (@ManyToOne и @OneToMany). Реализованы репозитории доступа к данным, сервисный слой с декларативным управлением транзакциями (@Transactional) и поддержкой аудита, а также применён архитектурный паттерн DTO для предотвращения циклических зависимостей и изоляции модели базы данных от внешнего REST API. Все автоматические интеграционные тесты успешно пройдены.